

Complete OOPs in C++ — From Basics to Advanced | Your Complete Placement Guide

29 min read · Aug 5, 2025



Sheel Ganvir



In this Medium article, I've compiled all the essential OOPs concepts that are commonly asked in interviews. After exploring various trusted resources, I've brought everything together into one comprehensive guide — from beginner to advanced level. It's a reliable resource to boost your on-campus placement preparation.

OOPS for OnCampus Placements

Beginner to Advanced

Complete OOPs in C++



Content of this blog

1. What is OOP?

- Object
- Class

2. What Is Padding or Structural Padding?

- Greedy Alignment

3. Access Modifiers in OOP

- Public
- Private
- Protected

4. Getters and Setters?

5. Object Creation in C++

- Static Allocation (stack)
- Dynamic Allocation (heap)

6. Constructor

- Default Constructor
- Parameterized Constructor
- Copy Constructor
- Shallow Copy vs Deep Copy

7. this Keyword

8. Destructor

- Static Allocation (Automatic Storage)
- Dynamic Allocation (heap)
- Garbage Collector

9. static keyword

- Static Data Member
- Static Member Function

10. Encapsulation

11. Abstraction

12. Inheritance

- Mode of Inheritance

13. Polymorphism

- Compile-time Polymorphism (function/ operator overloading)
- Run-time Polymorphism (Function Overriding)

14. Virtual Function

15. Types of Inheritance in C++

- Single Inheritance
- Multilevel Inheritance
- Multiple Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

16. Inheritance Ambiguity

- Scope Resolution Operator

17. Diamond Problem

- Virtual Inheritance.

18. Pointers, Inheritance, and Object Creation (Important for MCQ round)

19. Liskov Substitution Principle (LSP)

20. What is Low Coupling in OOP?

21. What is a Dangling Pointer?

22. What is an Abstract Class?

- Pure Virtual Function
- Interface vs Abstract Class

23. Early Binding (Static Binding) vs Late Binding (Dynamic Binding)

24. const keyword

25. final keyword

26. enum keyword
27. Aggregation – "Has-a" Relationship
28. Composition – "Has-a" Relationship
29. Generalization – "Is-a" Relationship
30. Struct vs Class
31. try-catch block
32. Error vs Exception
33. How many instances can be created for an abstract class?
34. Initialization
35. Default Arguments
36. Friend Function
37. Friend Class
38. Interview Problem on Friend Class
39. Procedural Language
40. Singleton Design Pattern

. . .

What is OOP?

OOP (Object-Oriented Programming) is a programming pattern that revolves around **objects** or **entities**.

It helps us:

- *Organize code better* 
- *Bundle data and functionality together* 

- Improve modularity and reusability 

Key Concepts

- **Object** = Entity having:
 - **State** ( **properties**) → Stored using *data members*
 - **Behaviour** ( **capabilities**) → Defined using *member functions*
- **Class** = A **blueprint** or template to create objects
It acts as a user-defined data type that holds data members and member functions.

Real C++ Example — Size of Class

```
#include <iostream>
using namespace std;

class CodeHelp {
    int subs; // int size = 4bytes
    bool a;   // bool size = 1 byte
};

int main() {
    cout << "Size of CodeHelp class in C++ is: " << sizeof(CodeHelp);
    return 0;
}
```

Size of CodeHelp class in C++ is: 8

But wait... 

- `int` = 4 bytes
- `bool` = 1 byte
- Total should be = 5 bytes, right?

But output is 8 bytes **!** Why??

Think of Memory Like Drawers

*Imagine memory as a row of drawers. Each data type needs a drawer of a **specific size** and likes to be placed in **certain positions**.*

Here's What Happens:

-  `int sub` takes 4 drawers →    
-  `bool a` takes 1 drawer → 
- Now we have used 5 drawers.

But here's the **trick**:

The **computer likes to work faster** when things are arranged in **neat rows** (multiples of 4 or 8). So, it adds **extra empty drawers** after `bool`.

. . .

What Is Padding or Structural Padding?

Padding = extra empty space added to keep things neat and fast for the computer.

So, after:

- `int sub` (4 bytes),
- `bool a` (1 byte),
 Computer adds 3 empty bytes to make total = **8 bytes**.

Why Is It Called Greedy Alignment?

Because the computer is being **greedy** and says:

“Let's keep the total size in **multiples of the biggest item** (which is 4 for int), even if it wastes a bit of space.”

✓ Final Memory Layout

Item	Size
<code>int sub</code>	4 bytes
<code>bool a</code>	1 byte
padding	3 bytes
Total	8 bytes

Super Simple Summary:

- ◆ **Padding** = extra space added by computer
- ◆ **Greedy alignment** = total size is made neat (like 4 or 8)
- ◆ Even if your class has 5 bytes of actual data, it may take **8 bytes** in memory
- ◆ It's done to make the program run **faster**.

Note : Size of empty class in C++ = 1 byte

. . .

Access Modifiers in OOP

Access modifiers control **who can access the data** in a class. Here's a quick breakdown:

 `public`

✓ Accessible from **anywhere** (inside or outside the class)

 *Used for methods or variables meant to be openly accessible.*

 `private`

✓ Accessible **only within the class**

 *Ensures **data hiding** and security.*

 ***Default in C++** if no modifier is specified.*

 `protected`

✓ Accessible within the class and derived classes

 Mostly used in *inheritance* scenarios.

In Java:

- `default` (package-private)
- `public`
- `private`
- `protected`

 **Interview Tip:**

“In C++, if no access modifier is specified, it defaults to `private`.” — a commonly asked MCQ or concept check!

Interview One-Liner:

“`private` members are hidden from everyone including derived classes, while `protected` members are accessible in derived classes — useful in inheritance.”

Bonus Tip (MCQ Trick):

Q: Can a derived class access private members of its base class?

A: ✗ No, only `protected` and `public` members are inherited accessibly.

. . .

 **What are Getters and Setters?**

Imagine a class like a sealed box . The data inside it (like `subs` or `views`) is **private** — meaning you can't access it directly from outside.

So how do we:

-  See what's inside? → Use a **getter**.
-  Change what's inside? → Use a **setter**.


```

class CodeHelp {
private:
    int subs;
    int views;

public:
    // Getter for subs
    int getSubs() {
        return subs;
    }

    // Setter for subs
    void setSubs(int value) {
        subs = value;
    }
};

int main() {

    CodeHelp ch; // ch --> object, CodeHelp --> Class

    ch.setSubs(100); // dot operator

    cout << "Subs = " << ch.getSubs() << endl;

    return 0;
}

```

✓ In Your Code:

```

private:
    int subs;
    int views;

```

These variables are **private** → Cannot be accessed directly from outside the class. So We Use:

■ Getter — to get the value:

```

int getSubs() {
    return subs;
}

```

```
}
```

■ Setter — to set the value:

```
int setSubs(int value) {  
    subs = value;  
}
```

🧠 Why Use Getters and Setters?

1. ✅ **Control access** — You can protect your data.
2. ✅ **Validation** — You can check if the new value is valid in the setter.
3. ✅ **Encapsulation** — Core concept of OOP that hides internal details.

🏷️ What Are Annotations?

► In Java / Python, annotations are special markers (starting with `@`) that give extra info to the compiler or runtime.

Examples:

- `@Override` in Java – tells the compiler a method is overriding a parent method.
- `@GetMapping` in Spring Boot (Java) – for defining API routes.
- `@property` in Python – turns a method into a getter.

But in C++, we don't have `@annotations` like Java or Python. Instead, we directly use **getter/setter methods**

. . .

◆ Object Creation in C++

There are 2 ways to create objects:

✅ 1. Static Allocation (Stack)

```
Hero a;    // object created statically
a.level = 5;
```

 **Stored in: Stack memory**

- Access members using: `.` dot operator
- Memory cleanup is automatically managed
- Values are usually **garbage (random)** if not initialized

✓ **2. Dynamic Allocation (Heap)**

```
Hero* b = new Hero();
b->level = 5;
```

 **Stored in: Heap memory**

- Access members using: `->` arrow operator
- You must manage memory (use `delete` if needed)
- Values are **initialized to 0** (default)

Bonus Memory Tip

`new` returns the **address** (i.e., a pointer). That's why we write `*`:

- `Hero* a = new Hero();`

Can we access data using dot operator (`.`) in dynamic allocation?

✓ **Yes!**

*In dynamic memory allocation, when we use a pointer to an object, we can access its members using the dot operator **after dereferencing**.*

 **Syntax:**

```
(*ptr).member
```

✨ Example:

```
(*ch).getSubs(); // ✅ Allowed
```

📌 This means:

- 📌 First, **dereference** the pointer using `*`.
- 📌 Then, **access the member** using the dot operator `.`

Note: Output will be `0` if the value is initialized to zero.

. . .

🔧 What is a Constructor in C++?

A constructor is a special member function of a class that:

- *Automatically executes when an object is created* 🧱
- *Initializes the data members of the class* 🎯
- *Has no return type* ❌ (not even `void`)
- *Has the same name as the class* 🏰

Does a constructor return any value in C++?

Ans. ✅ Yes — internally

A constructor doesn't have a return type, and we can't use the return keyword. But yes, it implicitly returns the object being created — handled by the compiler.

💬 Interview Tip:

Tasks Performed by a Constructor (Not Just Initialization)

“Besides initializing object state, a constructor can also acquire resources, validate inputs, and set up internal links.”

Constructor vs Method?

Constructors are special functions that initialize objects and run automatically when objects are created. Unlike methods, they have no return type, share the class name, and can't be called manually.

Types of Constructors (C++)

1. Default Constructor

- *No parameters*
- *Automatically created if no constructor is defined*

```
CodeHelp() { cout << "Default Constructor called"; }
```

2. Parameterized Constructor

- *Takes parameters to initialize object with values*

```
CodeHelp(int s, int v) {      subs = s;      views = v; }
```

3. Copy Constructor

- *Copies data from one object to another*

```
CodeHelp(CodeHelp &obj) {      subs = obj.subs;      views = obj.views; }
```

Can we make copy constructor private? Yes

Why is Copy Constructor Passed by Reference?

```
CodeHelp(CodeHelp &obj) { ... }
```

✓ Reason:

- If passed **by value**, it would again call the **copy constructor** to copy `obj`, leading to **infinite recursion** 
- Passing **by reference** avoids creating a new copy and breaks the cycle

Shallow Copy vs Deep Copy

1. Shallow Copy (Default behavior)

- Copies values of member variables
- For **pointer variables**, **only the address is copied**, not the data it points to
- Both objects **share same memory** (dangerous if one deletes it!)

```
obj1.ptr = new int(10);  
obj2 = obj1; // Both share same ptr → shallow copy
```

 If `obj1` is deleted, `obj2`'s pointer becomes *dangling*

2. Deep Copy

- Allocates **separate memory**
- Copies the **actual data**, not just pointers
- Ensures both objects are **independent**

```
CodeHelp(const CodeHelp &obj) {  
    this->ptr = new int(*obj.ptr); // Deep copy  
}
```

Example:

```

class CodeHelp {
public:
    int subs;
    int views;

    // ✅ Default Constructor
    CodeHelp() {
        subs = 0;
        views = 0;
        cout << "Default Constructor Called 🤖" << endl;
    }

    // ⚙ Parameterized Constructor
    CodeHelp(int s, int v) {
        subs = s;
        views = v;
        cout << "Parameterized Constructor Called ⚙" << endl;
    }

    // 📄 Copy Constructor
    CodeHelp(CodeHelp &obj) {
        subs = obj.subs;
        views = obj.views;
        cout << "Copy Constructor Called 📄" << endl;
    }

    // Method to display values
    void display() {
        cout << "Subscribers: " << subs << ", Views: " << views << endl;
    }
};

int main() {
    CodeHelp obj1;                // Default Constructor
    obj1.display();

    CodeHelp obj2(100, 2500);     // Parameterized Constructor
    obj2.display();

    CodeHelp obj3 = obj2;         // Copy Constructor
    obj3.display();

    return 0;
}

```

```

Default Constructor Called 🤖
Subscribers: 0, Views: 0

```

Parameterized Constructor Called ☼
Subscribers: 100, Views: 2500
Copy Constructor Called 📄
Subscribers: 100, Views: 2500

Compiler generates two constructor by itself.

1. Default Constructor
2. Copy Constructor.

But if any of the constructor is created by user, then default constructor will not be created by compiler. ➡

. . .

this Keyword in C++

🤔 What is this ?

- `this` is a **pointer** that points to the **current object** of the class.
- It is implicitly available in **non-static member functions**.

🎯 Why Use this ?

- To refer to **current object's members**
- To **resolve ambiguity** between **data members** and **constructor (or method) parameters**

When does ambiguity occur?

When *parameter names and class member names are identical*, C++ gets confused:

```
class CodeHelp {  
public:  
    int subs;  
  
    CodeHelp(int subs) {  
        subs = subs; // ❌ Ambiguous: both refer to parameter  
    }  
}
```



```
}  
};
```

👉 Above code **doesn't** assign to the class member!

✅ `this` Removes Ambiguity

```
class CodeHelp {  
public:  
    int subs;  
  
    CodeHelp(int subs) {  
        this->subs = subs; // ✅ Fixes ambiguity  
    }  
  
    void display() {  
        cout << "Subscribers: " << subs << endl;  
    }  
};
```

• • •

What is a Destructor in C++?

*A **destructor** is a special member function of a class that is **automatically called** when an object goes out of scope or is deleted.*

Its job? 🧹 To clean up resources (memory, files, connections) that your object used.

👉 Syntax:

```
~ClassName() {  
    // cleanup code  
}
```

E.g.,

```
~Codehelp() {  
    cout << "Destructor called" << endl;  
}
```

Static Allocation (Automatic Storage)

When you create an object **statically**, e.g.,

```
Codehelp a;
```

✅ The destructor is *called automatically* when `a` goes out of scope (like exiting the function/block).

Example Flow:

```
{  
    Codehelp a;  
    // do something with a  
} // 🛎️ Destructor called automatically here!
```

✅ You don't need to do anything manually!

Dynamic Allocation (Heap Storage)

When you create an object using `new`, e.g.,

```
Codehelp* d = new Codehelp();
```

😬 C++ does *not* automatically destroy it when you're done.

👉 You *must* manually delete it:

```
delete d; // 🚨 Destructor is called here!
```

If you forget `delete`, you get a memory leak!

What About Java?

Java has a Garbage Collector 🗑️:

- It automatically frees memory when no references point to an object.
- You don't call `delete`.
- You don't have destructors; instead, you have **finalizers** (but they're rarely used).

✅ In Java:

```
MyClass obj = new MyClass();  
// no need to delete!
```

❌ Why No Garbage Collector in C++?

- C++ was designed for *performance and control*.
- *Garbage Collector* = automatic memory cleanup.
- C++ believes you, the programmer, know best when to free memory.
- Adding garbage collection slows things down, so it's left out.

How to Delete Dynamically Allocated Arrays?

When you allocate an array with `new[]`:

```
int* arr = new int[5];
```

✅ You must delete it using `delete[]`:

```
delete[] arr;
```

. . .

What is `static` keyword in OOP?

In C++, the `static` keyword can be used **inside classes** to:

Types of `static` Members

Static Data Member

 Shared variable for all objects

- *Only **one copy exists**, no matter how many objects you create.*
- *Memory is allocated **once**, when the program starts.*

Use case:

*To keep **common information**, like a counter tracking the number of objects.*

Example:

```
class Student {
public:
    static int count; // static data member
    Student() {
        count++;
    }
};

// Initialize static member outside class
int Student::count = 0;
int main() {
    Student s1;
    Student s2;
    cout << "Total students: " << Student::count << endl;
}
```

Output:

Total students: 2

💡 Notice: We accessed it as `Student::count` without an object!

2 Static Member Function

📌 Function that can be called without an object

- Can only access *static data members*, not non-static ones.
- No `this` pointer.

✅ Use case:

To work with static data or utility functions.

Example:

```
class MyClass {
    public:
        static void showMessage() {
            cout << "Hello from static function!" << endl;
        }
};

int main() {
    MyClass::showMessage(); // called without any object
}
```

. . .

The 4 Pillars of OOP

1 Encapsulation

Binding data and functions together.

✅ Keeps data safe from outside interference.

✅ You access or modify data through methods (getters/setters).

💡 Example:

```
class Student {
private:
    int age; // hidden data
public:
    void setAge(int a) {
        age = a;
    }
    int getAge() {
        return age;
    }
};

int main() {
    Student s;
    s.setAge(20);
    cout << "Age: " << s.getAge();
}
```

✨ Why?

- *Protects data.*
- *Controls access.*

2 Abstraction

Showing only essential details and hiding complexity.

- ✓ *Makes code easier to use.*
- ✓ *You don't need to know how it works inside.*

💡 Example:

```
class Car {
public:
    void start() {
        cout << "Car started 🚗";
    }
    void stop() {
        cout << "Car stopped 🛑";
    }
};
```

```
int main() {  
    Car c;  
    c.start(); // You don't see engine internals!  
}
```

✨ Why?

- *Hides complexity.*
- *Provides simple interface.*

3 Inheritance

Deriving a new class from an existing class.

✓ Reuse code.

✓ Create a “is-a” relationship.

💡 Example:

```
// Base Class / Parent Class  
class Animal {  
public:  
    void eat() {  
        cout << "Eating...";  
    }  
};  
  
// Derived Class / Child Class  
// Mode of Inheritance: public  
class Dog : public Animal {  
public:  
    void bark() {  
        cout << "Woof!";  
    }  
};  
  
int main() {  
    Dog d;  
    d.eat(); // Inherited from Animal  
    d.bark(); // Dog's own function  
}
```

✨ Why?

- *Reusability.*
- *Extensibility.*

Mode of Inheritance:



Base class member access specifier	MODE OF INHERITANCE		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not Accessible (Hidden)	Not Accessible (Hidden)	Not Accessible (Hidden)

4 Polymorphism

Poly → many and morphism → forms i.e., exists in multiple forms. Same interface, different behavior.

✓ Many forms.

👉 Two types:

- Compile-time (function/ operator overloading)
- Run-time (virtual functions)

Example: Function Overloading

```
class Math {  
public:  
    int sum(int a, int b) {  
        return a + b;  
    }  
}
```



```

    int sum(int a, int b, int c) {
        return a + b + c;
    }
};

int main() {
    Math m;
    cout << m.sum(2, 3);          // 5
    cout << m.sum(1, 2, 3);      // 6
}

```

Example: Operator Overloading

*Operator Overloading means giving **custom meaning** to an operator (+, -, *, etc.) for user-defined types (like classes).*

You want to add two Hero objects using the + operator:

```
Hero ans = a + b;
```

*But C++ doesn't know how to add two Hero objects unless **you define** what + should mean for them. That's where **operator overloading** comes in!*

```

Hero operator+(Hero b) {
    Hero ans;
    ans.age = this->age + b.age;
    ans.weight = this->weight + b.weight;
    ans.height = this->height + b.height;
    return ans;
}

```

❌ Operators That CANNOT be Overloaded

Operator	Reason
::	Scope resolution
.	Member access
.*	Pointer-to-member
?:	Ternary/conditional

🤔 Why << and >> Overloading Must Be Outside Class?

These are **stream operators** used with `cout` and `cin`.

*Since `cout` is not a part of your class, overloading must be done **outside** the class using **friend functions**.*

You want to do:

```
cout << heroObj;
```

But by default, `cout` doesn't know how to print your **custom class object**.

So we need to teach it — by **overloading the << operator**.

Steps to Overload <<:

👉 1. Inside the class — declare it as friend:

```
class Hero {
public:
    int age;
    int weight;
    int height;
    // Friend function declaration
    friend ostream& operator<<(ostream &out, Hero &h);
};
```

2. Outside the class — define the function:

```
ostream& operator<<(ostream &out, Hero &h) {  
    out << h.age << " " << h.weight << " " << h.height;  
    return out;  
}
```

```
Hero a;  
a.age = 10;  
a.weight = 20;  
a.height = 30;  
  
cout << a; // prints: 10 20 30
```

Example: Run-time Polymorphism (Function Overriding)

```
class Animal {  
public:  
    virtual void sound() {  
        cout << "Animal sound";  
    }  
};  
  
class Dog : public Animal {  
public:  
    void sound() override {  
        cout << "Woof!";  
    }  
};  
  
int main() {  
    Animal* a;  
    Dog d;  
    a = &d;  
    a->sound(); // "Woof!" (dynamic binding)  
}
```

What is a Virtual Function in C++?

A virtual function is a function in a base class that you expect to override in derived classes.

💡 Declared using the keyword `virtual` in the base class.

✨ Why?

- *Flexibility.*
- *One interface, multiple behaviors.*

⚠️ Limitations of Virtual Functions

Slower execution — Resolved at runtime (not compile-time).

More memory — Uses vtable and vptr.

No polymorphism in constructor/destructor — Always calls base version.

Tightly coupled — Requires inheritance.

Hard to debug — Runtime errors possible if not handled properly.

• • •

Types of Inheritance in C++

Inheritance in C++

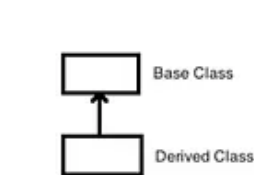


Fig. Single level inheritance

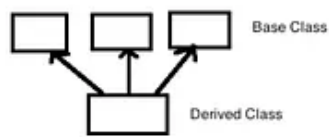


Fig. Multiple Inheritance

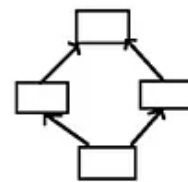


Fig. Hybrid Inheritance

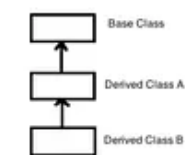


Fig. Multi level inheritance

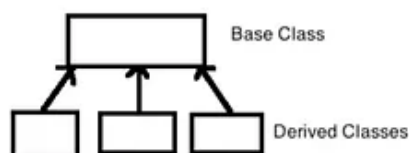


Fig. Hierarchical inheritance

1 Single Inheritance

One base class, one derived class.

Simplest form.

Example:

```
class Parent {  
public:  
    void show() { cout << "Parent"; }  
};  
  
class Child : public Parent {};
```

2 Multilevel Inheritance

A class derived from another derived class.

Chain of inheritance.

Example:

```
class Grandparent {  
public:  
    void show() { cout << "Grandparent"; }  
};  
  
class Parent : public Grandparent {};  
class Child : public Parent {};
```

3 Multiple Inheritance

One class inherits from more than one base class.

Combines features from multiple parents.

Example:

```
class Parent1 {  
public:  
    void show1() { cout << "Parent1"; }  
};  
  
class Parent2 {
```

```
public:
    void show2() { cout << "Parent2"; }
};
class Child : public Parent1, public Parent2 {};
```

4 Hierarchical Inheritance

Multiple derived classes inherit from the same base class.

Example:

```
class Parent {
public:
    void show() { cout << "Parent"; }
};

class Child1 : public Parent {};
class Child2 : public Parent {};
```

5 Hybrid Inheritance

Combination of more than one type of inheritance.

Often involves *multiple* and *hierarchical* inheritance.

Example:

```
class A {
public:
    void showA() { cout << "A"; }
};

class B : public A {};
class C {
public:
    void showC() { cout << "C"; }
};
class D : public B, public C {};
```

• • •

Inheritance Ambiguity

When does ambiguity happen?

=> When **multiple** base classes have members with the **same name**, and the compiler **doesn't know which one you mean**.

Example: Multiple Inheritance Ambiguity

```
class A {
public:
    void show() {
        cout << "A's show()" << endl;
    }
};

class B {
public:
    void show() {
        cout << "B's show()" << endl;
    }
};

class C : public A, public B {
};

int main() {
    C obj;
    obj.show(); // ❌ Ambiguity! Compiler error.
}
```

😞🤔 Why error?

- `C` inherits *two* `show()` functions, one from `A`, one from `B`.
- The compiler doesn't know which to call.

✅ How to solve it? Scope Resolution Operator

Scope resolution operator `::` lets you specify exactly which base class's method you want.

Example fixing ambiguity:

```
int main() {  
    C obj;  
    obj.A::show(); // Calls A's show()  
    obj.B::show(); // Calls B's show()  
}
```

✓ Output:

```
A's show()  
B's show()
```

Quick Notes on Scope Resolution Operator ::

✓ It is used for:

- Accessing *members from specific base classes to resolve ambiguity.*
- Defining a function *outside the class:*

```
class MyClass {  
public:  
    void display();  
};  
  
void MyClass::display() {  
    cout << "Hello!";  
}
```

- Accessing *global variables when local variable hides them:*

```
int x = 10; // global  
  
int main() {  
    int x = 20; // local
```



```
cout << ::x; // prints 10
}
```

. . .

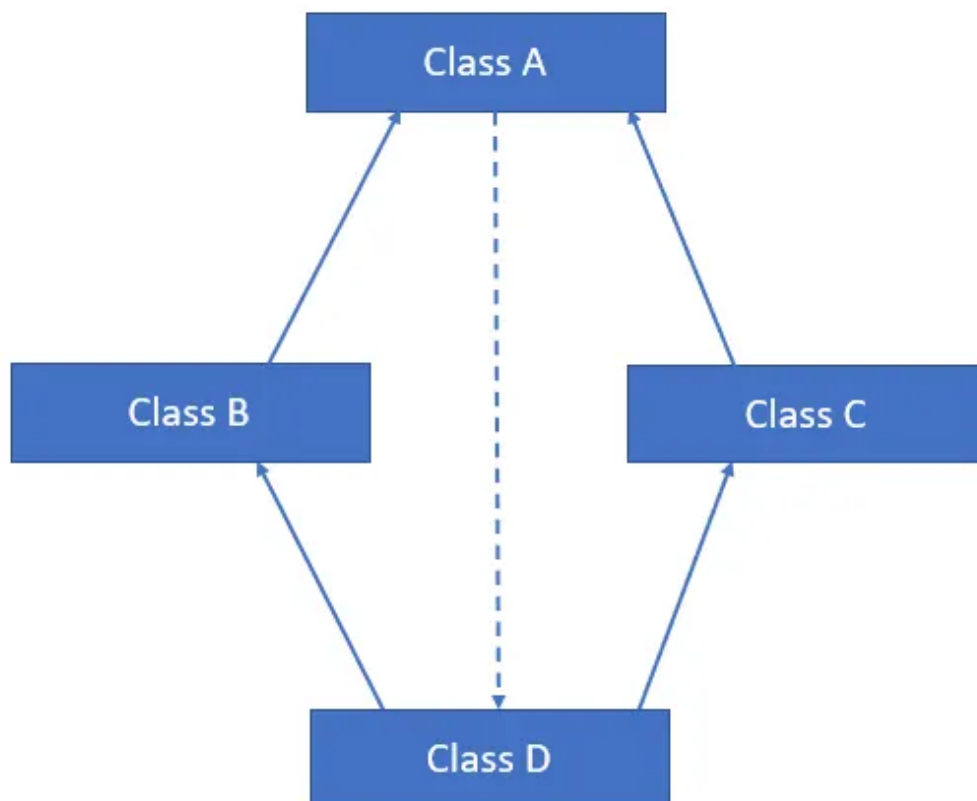
💎 Diamond Problem in C++

✅ What is it?

It happens when:

- A class inherits from two classes,
- Both of those classes inherit from the same base class.

This forms a **diamond-shaped inheritance diagram**:



Why is it a problem?

👉 Because D inherits **two copies** of A's members (through B and C). This leads to **ambiguity** and **data duplication**.

Example of Diamond Problem

```

class A {
public:
    void show() {
        cout << "Class A\n";
    }
};
class B : public A {
};
class C : public A {
};
class D : public B, public C {
};
int main() {
    D obj;
    obj.show(); // ❌ Error: 'show' is ambiguous
}

```

😞🤔 Why error?

- `D` has *two copies of `A::show()`* (one from `B` and one from `C`).
- Compiler doesn't know which one you mean.

How to Solve It?

✅ Use **Virtual Inheritance**.

Virtual inheritance makes sure there is only **one shared base A** no matter how many paths you inherit it through.

● Solution Example (Virtual Inheritance)

```

class A {
public:
    void show() {
        cout << "Class A\n";
    }
};
// Use 'virtual' keyword
class B : virtual public A {
};
class C : virtual public A {
};

```

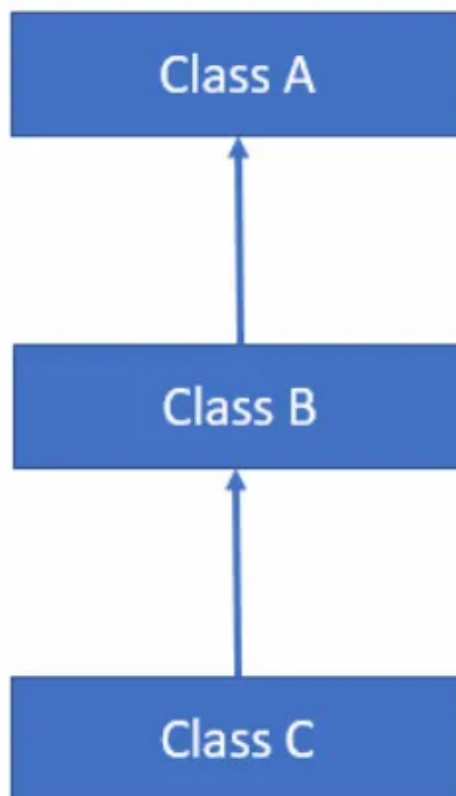
```
class D : public B, public C {
};
int main() {
    D obj;
    obj.show(); // ☒ No ambiguity
}
```

Now *only one A* exists in `D`. Ambiguity is resolved.

• • •

Pointers, Inheritance, and Object Creation (Important for MCQ round)

Suppose:



```
class A { };
class B : public A { };
class C : public B { };
```

✓ Valid Pointer Assignments

`A* obj = new A();` ✓ Fine — base class pointer to base object

`A* obj = new B();` ✓ Fine — base pointer to derived object (upcasting)

`B* obj = new C();` ✓ Fine — base pointer to derived object (upcasting)

Upcasting is always safe and allowed implicitly.

✗ Invalid Pointer Assignments

`B* obj = new A();` ✗ Not allowed — can't assign base object to derived pointer (downcasting)

`C* obj = new A();` ✗ Not allowed — downcasting without cast

Downcasting is not allowed without explicit casting, and it's unsafe.

Q1: Which of the following is valid?

✓ `A* obj = new C();`

✓ `B* obj = new C();`

✗ `C* obj = new A();`

Q2: Why is `A* obj = new B();` allowed?

👉 Because in inheritance, a **base class pointer** can point to a **derived class object**.

Q3: What's the use of such assignment?

👉 To achieve **polymorphism** (e.g., virtual functions).

Diagram on the Right:

Shows class `C` → inherits `B` → inherits `A`

So:

✓ `A* obj = new C();` is valid because `C` IS-A `B`, and `B` IS-A `A`.

. . .

Liskov Substitution Principle (LSP)

Definition:

If class `S` is a subclass of class `T`, then objects of type `T` should be replaceable with objects of type `S` **without altering the correctness** of the program.

In Simple Words:

You should be able to use a **derived class** object anywhere a **base class** object is expected — **without breaking anything!**

Real-life Example:

```
class Bird {
public:
    virtual void fly();
};

class Sparrow : public Bird {
    void fly(); // ✓ Valid
};

class Ostrich : public Bird {
    void fly(); // ✗ Violates LSP - Ostriches can't fly!
}
```

Here, replacing `Bird* b = new Ostrich();` will break logic if the code expects all birds to fly — hence **LSP is violated**.

Interview One-Liner:

LSP ensures that a subclass behaves in a way that doesn't surprise or break the expectations set by its superclass.

Key Rule for MCQs:

If a derived class **overrides a method** but **changes expected behavior**, it's a violation of LSP

• • •

What is Low Coupling in OOP?

Low Coupling means that **classes or modules are independent and not tightly connected** to each other.

✓ They **only know what they need to know** — nothing more.

“If changing one class doesn’t force changes in another, they have **low coupling**.”

. . .

What is a Dangling Pointer?

A **dangling pointer** is a pointer that **still points to a memory location** which has already been **freed or deleted**.

Accessing it leads to **undefined behavior**. *Can cause crashes*

Example:

```
int* ptr = new int(10); // dynamic allocation
delete ptr;             // memory freed

cout << *ptr; // ✗ Dangling pointer! Accessing freed memory
```

How to Avoid Dangling Pointers?

Set pointer to `nullptr` after `delete`

```
delete pt
ptr = nullptr; // ✓ Safe now
```

. . .

What is an Abstract Class?

An *abstract class* in C++ is a class that *cannot be instantiated directly*.
It serves as a *blueprint* for derived classes.

How to make a class abstract?

Use a **pure virtual function** in the base class:

```
class Hero {
public:
    virtual void dummy() = 0;  // 🖱️ Pure virtual function
};
```

This makes `Hero` an **abstract class**.

What is a Pure Virtual Function?

A **pure virtual function** is a virtual function declared like this:

```
virtual void func() = 0;
```

- It has **no body** in the base class.
- **Must** be overridden in the derived class.
- Forces derived class to implement the method.

Makes the class **abstract** — meaning, you *can't* create an object of this class.

1st Example (Abstract Class with Pure Virtual Function)

```
class Hero {
public:
    virtual void dummy() = 0;  // Pure virtual
};

class Player : public Hero {
public:
    void dummy() {
        cout << "I am in player class";
    }
};

Hero* obj = new Player();
obj->dummy();
```

Output: I am in player class

Explanation:

- `Hero` is abstract (has pure virtual function).
- `Player` overrides `dummy()` → so it can be instantiated.
- `Hero*` can point to `Player` due to polymorphism.

✗ *If you don't override `dummy()` in `Player`:*

You will get an error:

“cannot instantiate abstract class ‘Player’”

2nd Example (No virtual, both classes have `dummy()`)

```
class Hero {
public:
    void dummy() {
        cout << "I am in hero class";
    }
};

class Player : public Hero {
public:
    void dummy() {
        cout << "I am in player class";
    }
};

Hero* obj = new Player();
obj->dummy();
```

Output: I am in hero class

Why?

- No `virtual` = no runtime polymorphism.
- Function binding happens at **compile-time**.
- Since pointer is of type `Hero*`, it calls `Hero::dummy()`.

3rd Example (Virtual function, overridden in derived)


```

class Hero {
public:
    virtual void dummy() {
        cout << "I am in hero class";
    }
};

class Player : public Hero {
public:
    void dummy() {
        cout << "I am in player class";
    }
};

Hero* obj = new Player();
obj->dummy();

```

Output: I am in player class

Why?

- `dummy()` is `virtual`, so runtime polymorphism kicks in.
- Derived class `Player` overrides it.
- So, `Player::dummy()` is called.

Interface vs Abstract Class in C++

◆ Abstract Class:

```

class Shape {
public:
    virtual void draw() = 0; // pure virtual
    void commonMethod() { cout << "Shared"; }
};

```

◆ Interface:

An interface in C++ is a class with only pure virtual functions, used to achieve complete abstraction.

```
class IShape {
public:
    virtual void draw() = 0;
    virtual void resize() = 0;
    // No data members, no method body
};
```

One-Liner for Interviews:

Abstract classes allow partial abstraction and shared code, while **interfaces** provide complete abstraction with only method declarations.

. . .

Early Binding (Static Binding)

- Happens at compile time
- Used in: Normal function and Function overloading
-  **Faster** (no extra overhead)
-  No runtime flexibility

Example:

```
class A {
public:
    void show() { // not virtual
        cout << "A";
    }
};

A obj;
obj.show(); // Early binding (at compile time)
```

Late Binding (Dynamic Binding)

- Happens at runtime

- Used in: **Virtual functions**
-  Allows **runtime polymorphism**
-  Slightly **slower** due to vtable

Example:

```
class A {
public:
    virtual void show() {
        cout << "A";
    }
};

class B : public A {
public:
    void show() {
        cout << "B";
    }
};

A* obj = new B();
obj->show(); // Late binding → calls B::show()
```

Interview One-Liner:

Early binding is compile-time function linking, while **late binding** uses `virtual` functions to resolve function calls at runtime.

. . .

★ What is `const` ?

`const` is like putting a "Do Not Touch"  label on a variable, method, object, or pointer.

It means "You cannot change this value after it is set."

`const` with variables

```
const int a = 5; // ❌ Can't change later
a = 10; // ❌ Error: Read-only variable
```

2 `const` with pointers

Here's the tricky part — pointer + const has 3 cases:

Case 1: `const int* a`

```
int x = 5;
int y = 12;
const int* a = &x;

*a = 10; // ❌ Can't change value through pointer
a = &y; // ✅ Can make pointer point to another variable
```

Meaning: Data is constant, pointer can move.

Case 2: `int* const a`

```
int x = 5;
int y = 12;
int* const a = &x;

*a = 10; // ✅ Can change value through pointer
a = &y; // ❌ Can't change where pointer points
```

Meaning: Pointer is constant, data can change.

Case 3: `const int* const a`

```
int x = 5;
const int* const a = &x;
```

```
*a = 10;    // ❌ Can't change value  
a = &y;     // ❌ Can't change pointer
```

Meaning: Both pointer and data are constant.

3 `const` with member functions in OOP

When you write:

```
void print() const;
```

It means **this function will not modify any data members of the class.**

Example:

```
class Player {  
public:  
    int score;  
  
    void showScore() const {  
        // score = 100; ❌ Not allowed (can't modify inside const function)  
        cout << score;  
    }  
};
```

. . .

🌟 What is `final`?

The `final` keyword in C++ is like saying:

“This is the last stop. No more changes allowed here!” 🚫

It is used in two main ways:

1 Prevent Method Overriding

If you put `final` after a **virtual function**, it means **derived classes cannot override it**.

Example:

```
class Base {
public:
    virtual void show() final { // ❌ Can't override in child
        cout << "Base show()" << endl;
    }
};

class Derived : public Base {
public:
    void show() { // ❌ Error! Can't override because it's final
        cout << "Derived show()" << endl;
    }
};
```

2 Prevent Class Inheritance

If you put `final` after a **class name**, it means **no other class can inherit from it**.

Example:

```
class Base final { // ❌ Can't inherit from this class
public:
    void display() {
        cout << "Base class" << endl;
    }
};

class Derived : public Base { // ❌ Error! Can't inherit
};
```

*Use this when you want a class to be **the last in the inheritance chain**.*

. . .

🌟 What is `enum`?

`enum` (short for **enumeration**) is a **user-defined data type** that lets you give **names** to a set of related values.

Think of it like **labeling numbers** so you don't have to remember them.

✂ How it works:

- The values inside an `enum` are automatically assigned **integers** starting from `0` by default.
- You can also **set custom values**.

Example:

```
enum Week { Mon = 1, Tue, Wed, Thu, Fri, Sat, Sun };  
  
cout << Mon; // Output: 1  
cout << Wed; // Output: 3
```

. . .

1 Aggregation — "Has-a" Relationship (Loosely Connected)

💡 **Meaning:** One object has another object, but they can live separately.

Example:

- *A teacher works in a school.*
- *If the school closes, the teacher can still work somewhere else.*

```
class Teacher {  
public:  
    string name;  
};  
  
class School {  
public:  
    Teacher teacher; // school has a teacher  
};  
// 📌 The teacher exists even without the school
```

. . .

2 Composition — “Has-a” Relationship (Strongly Connected)

💡 **Meaning:** One object is made up of another object, and they **cannot** be separated.

Example:

- *A car has an engine.*
- *If the car is destroyed, the engine is useless.*

```
class Engine {
public:
    string model;
};

class Car {
public:
    Engine engine; // car is made of engine
};
// 📌 The engine only exists because the car exists.
```

. . .

3 Generalization — “Is-a” Relationship

💡 **Meaning:** Child class is a **more specific version** of a parent class.

Example:

- *A dog is an animal.*
- *Dog inherits features of Animal.*

```
class Animal {
public:
    void eat() {}
};

class Dog : public Animal {
public:
```



```
void bark() {}  
};  
  
// 📌 Dog is-a Animal.
```

• • •

Struct vs Class

☀ In C++: Structs and Classes Are Almost the Same

The only real difference by default is:

Feature	struct	class
Default access	Public	Private
Default inheritance	Public	Private

Everything else — constructors, destructors, inheritance, member functions — works in both. ✅

📌 When to Use struct

Use struct when:

1. All or most data is public 📁
→ Example: Simple data containers.
2. You just want to **group related variables** without much logic.

```
struct Point {  
    int x;  
    int y;  
};
```

1. For *POD (Plain Old Data)* types → where you store data without complex behavior.

2. For *lightweight, passive objects used in algorithms (e.g., coordinate points, color values)*.

When to Use `class`

Use `class` when:

1. You need data hiding / encapsulation  (private members).
2. You have **complex behavior + data** combined.

```
class BankAccount {  
private:  
    double balance;  
  
public:  
    void deposit(double amount) {  
        balance += amount;  
    }  
};
```

1. You are designing *OOP models with inheritance, polymorphism, etc.*
2. You need *control* over how the object is used.

. . .

try-catch block

In C++, `try-catch` is used for **exception handling** — a way to handle runtime errors without crashing the program.

Basic syntax:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    try {  
        int x = 0;  
        if (x == 0)
```

```

        throw "Division by zero error!";
    cout << 10 / x;
}
catch (const char* msg) {
    cout << "Exception caught: " << msg;
}
}

```

- **try** → contains code that might cause an error.
- **throw** → signals an exception.
- **catch** → handles the thrown exception.

• • •

Error vs Exception

Feature	Error 🚫 (System Level)	Exception ⚠️ (Program Level)
Cause	Serious problems like memory overflow, hardware crash	Issues during program execution like invalid input, division by zero
Handling	Usually not recoverable in code	Can be handled with try-catch
Example	Stack overflow, segmentation fault	File not found, out-of-range index

In C++, **exceptions** are what **try-catch** deals with. **Errors** usually stop the program and cannot be caught.

• • •

3. How many instances can be created for an abstract class?

In C++:

- **Abstract class** → A class with at least **one pure virtual function**.
- **Pure virtual function** → `virtual void func() = 0;`

📌 **Rule:** You **cannot** create **any** direct instance of an abstract class.

But you **can** create instances of derived classes that **override all pure virtual functions**.

Example:

```
class Shape {
public:
    virtual void draw() = 0; // pure virtual
};
class Circle : public Shape {
public:
    void draw() { cout << "Drawing Circle\n"; }
};
int main() {
    // Shape s; ❌ Not allowed
    Circle c;    // ✅ Allowed
    c.draw();
}
```

Answer: 0 instances of an abstract class can be directly created

. . .

1 Initialization in OOP (C++)

Think of **initialization** as giving your variables their first value when they are born 🎂.

In **OOP**, we often give values to **object data members** when we create the object. We do this using **constructors**.

📦 Example — Without Initialization

```
#include <iostream>
using namespace std;

class Student {
    string name;
    int age;
public:
    Student(string n, int a) { // Constructor
        name = n; // initializing variables
        age = a;
    }
}
```

```

void display() {
    cout << "Name: " << name << ", Age: " << age << endl;
}
};
int main() {
    Student s1("Sheel", 21); // Object creation + initialization
    s1.display();
}

```

💡 Here:

- "Sheel" and 21 are given **right at birth** of the object.
- Constructor is like a **delivery box** 📦 that brings your object into the world with values already inside.

Initialization List (shortcut way)

Instead of:

```

name = n;
age = a;

```

We can do:

```

Student(string n, int a) : name(n), age(a) {}

```

💡 *This is **faster** because values are directly placed in the box at birth rather than putting them later.*

2 Default Arguments in OOP (C++)

Default argument = “If you don’t give me a value, I’ll use my backup value.”

📦 Example:

```
#include <iostream>
using namespace std;

class Student {
    string name;
    int age;
public:
    Student(string n = "Unknown", int a = 18) { // default values
        name = n;
        age = a;
    }
    void display() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};

int main() {
    Student s1; // No values given → uses defaults
    Student s2("Sheel", 21); // Values given → uses them
    s1.display(); // Name: Unknown, Age: 18
    s2.display(); // Name: Sheel, Age: 21
}
```

. . .

Friend Function

A **friend function** is a normal function that's allowed to access the **private** and **protected** members of a class.

 In your second code:

```
class Math {
private:
    int value = 150;

friend void printValue(Math n); // friend function declaration
};

void printValue(Math temp) {
    cout << temp.value << endl; // can access private member
}
```

- Normally, `value` is private and can't be accessed outside the class. But because `printValue` is declared as a **friend**, it's allowed to peek inside and access `value`.

2 Friend Class

A **friend class** means that all member functions of another class get access to your class's **private** and **protected** members.

📌 In your first code:

```
class A {
private:
    int value;
    void print() {
        cout << "I am in A class" << endl;
    }

friend class B; // class B is a friend
};
class B {
public:
    void printValue(A temp) {
        temp.print(); // can call A's private method
    }
};
```

Normally, `B` wouldn't be able to call `A`'s private method `print()`. But since `B` is declared as a friend, **all** its methods can access `A`'s private/protected stuff.

• • •

Question:

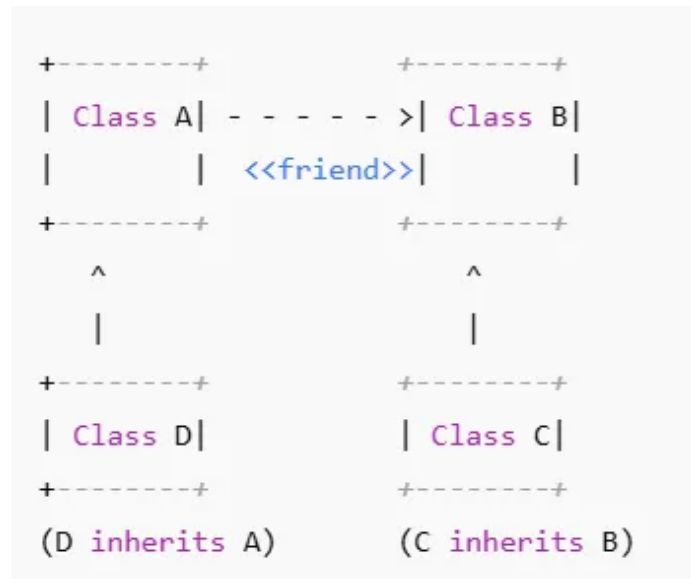
If we have:

- `Class B` is declared as a **friend** of `Class A`.
- `Class C` inherits from `Class B`.
- `Class D` inherits from `Class A`.

Then:

1. Can we say **B** is a friend of **D**?

2. Can we say **C** is a friend of **A**?



Scenario:

1. `class B` is a **friend** of `class A`.

- `class A { friend class B; };`

1. `class C` **inherits** `class B`.

2. `class D` **inherits** `class A`.

Question 1: Is B a friend of D?

- No **✗**.
- Friendship is **not inherited downward** through base-derived relationships.
- Even though `B` can access `A`'s private/protected members, it does not automatically get access to `D`'s members — unless `D` explicitly declares `B` as a friend.
- Why? Because `D` is a *different* class; inheritance doesn't copy friendship rights.

Question 2: Is C a friend of A?

- No **✗**.
- Friendship is **not inherited upward** through derived classes either.

- Just because `B` is a friend of `A` does not mean any class derived from `B` (`C` here) is also a friend of `A`.
- If you want `C` to be a friend of `A`, you must explicitly declare it as a friend inside `A`.

💡 Key Rule to Remember

Friendship in C++ is neither inherited nor transitive.

- *Not inherited* → A friend's child class doesn't get friendship automatically.
- *Not transitive* → If `B` is friend of `A` and `C` is friend of `B`, that doesn't mean `C` is friend of `A`.

• • •

Procedural Language

A **procedural language** is a programming paradigm based on the concept of **procedure calls** (also known as routines, subroutines, or functions). In this approach, the program is organized into a set of instructions executed **step-by-step** in a sequence.

It focuses on **how to do** a task rather than **what to do**.

✂ **Examples:** C, Pascal, Fortran, BASIC

🔑 **Key features:**

- 📄 **Top-down approach**
- 📁 **Code divided into functions/procedures**
- 🔄 **Emphasis on sequence, selection, iteration**
- 📊 **Data and functions are separate**
- 📦 **State changes via global/local variables**

Structured/Procedural Programming	Object Oriented Programming
Code is divided into modules or functions	Code is made up of classes and objects
Top-down approach	Bottom-up approach
Difficult to modify / manage	Easy to modify / manage
Main function calls other functions	Objects communicate by passing messages
Data is not secured	Data is secure
Less reusability of code	More reusability of code
Less flexibility and abstraction	More flexibility and abstraction

. . .

Singleton Design Pattern

Ensures only one object of a class is created in the whole program and provides a global access point to it.

Short Example

```
#include <iostream>
using namespace std;

class Bell {
private:
    static Bell* onlyBell; // the single bell
    Bell() { cout << "🔔 Bell created!\n"; } // private constructor

public:
    // If constructor is private:
    // We can't create Bell object outside the class,
    // but we can create it inside the class using 'static' keyword.
    static Bell* getBell() {
        if (onlyBell == nullptr) { // if no bell exists yet
            onlyBell = new Bell(); // object created inside class
        }
        return onlyBell; // return same bell
    }
}
```

```
void ring() { cout << "Ring Ring! 🛎️\n"; }  
};  
  
// initialize static variable  
Bell* Bell::onlyBell = nullptr;  
  
int main() {  
    Bell* teacher1 = Bell::getBell();  
    teacher1->ring();  
  
    Bell* teacher2 = Bell::getBell();  
    teacher2->ring();  
  
    cout << "teacher1 bell address: " << teacher1 << "\n";  
    cout << "teacher2 bell address: " << teacher2 << "\n";  
}
```

Output:

🛎️ Bell created!

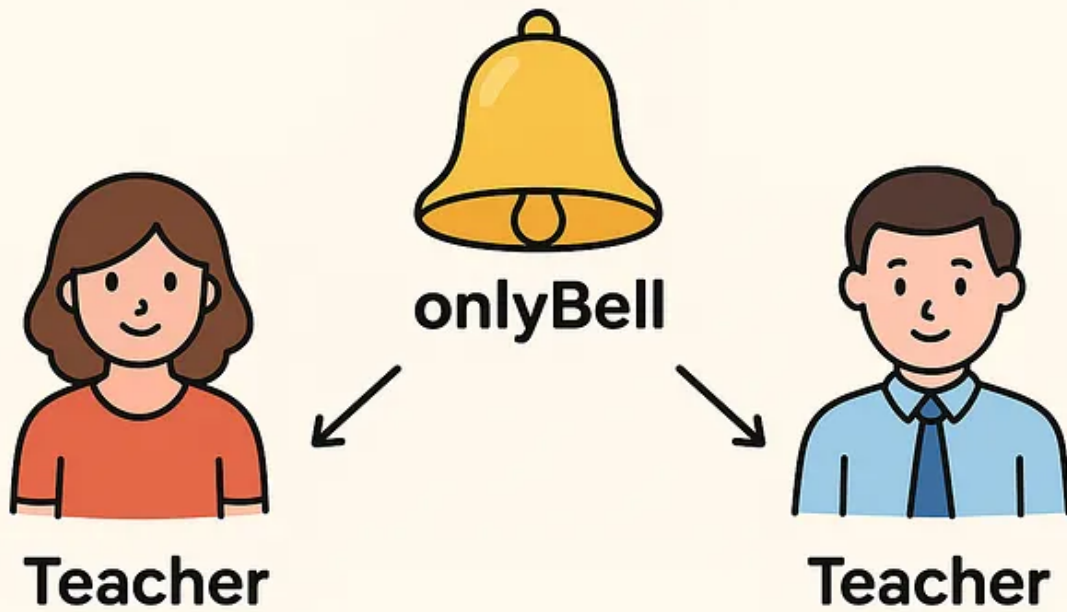
Ring Ring! 🛎️

Ring Ring! 🛎️

teacher1 bell address: 0x562f4b3e4eb0

teacher2 bell address: 0x562f4b3e4eb0

Singleton



🔑 Key Points

- 🔒 **Private constructor** → Stops creating objects outside the class.
- 📦 **Static member** → Stores the single instance inside the class.
- 📞 **Static method** → Only way to get that instance.
- 🚫 **Delete copy & assignment** → No accidental duplicates.
- ♻️ **Same object reused** → Saves memory & ensures consistency.

📌 When to Use

- 📄 **Logger system**
- 🗄️ **Database connection**
- ⚙️ **Config manager**
- 🎮 **Game settings**

Common Interview Q&A

Q1: What is Singleton pattern?

A: Restricts a class to *one instance* and provides a *global point of access*.

Q2: Why private constructor?

A: Prevents creating objects from outside the class — object is only created **inside** using a static member.

Q3: How to prevent multiple copies?

A: Delete the copy constructor & assignment operator.

Q4: Is it thread-safe?

A: Basic version ❌ not thread-safe. Use mutex or C++11 `static` local initialization ✅.

. . .

Congratulations! 🎉

If you've made it this far, you should not miss a similar computer networking article.

Click the link below 📌

Computer Networking article: [Click here](#)

If you found this article helpful, don't forget to **share it with your friends** who are also preparing.

If you liked my efforts and want to support it then you can donate any amount on the below:

- **For Indian Users:** Pay via UPI => sheelganvir2805@oksbi
- **For non-Indian Users:** <https://buymeacoffee.com/sheelganvir>

You can connect me on:

[LinkedIn](#) | [Medium](#) | [Github](#) | [X](#) | [YouTube](#) | [Instagram](#) | [Email](#)

Got doubts or suggestions? Drop them in the **comments** — let's learn and grow together!

Happy Coding & All the Best for Your Placements! ✨

Oops Concepts

Oops Interview Questions

Placement

Campus Placements

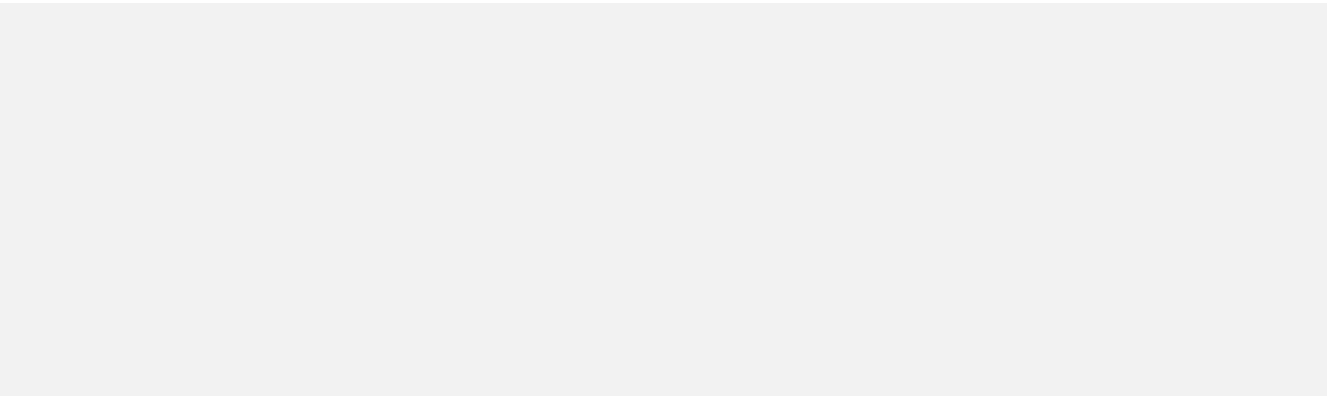
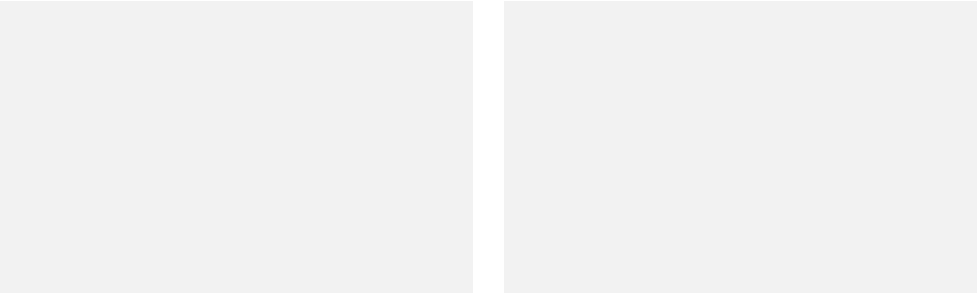
Cpp

Written by Sheel Ganvir

59 followers · 6 following

Software Engineer | Writing some free stuffs which can be valuable!!





[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]